

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Branch: CSM

Section: B

Task No.: 6

Student Name: M. Niharika

Roll No.: A24126552097

1. TITLE

Implement Modules in Node.js — Built-in and User-Defined Modules

2. AIM

To understand and implement **modules in Node.js** by using several built-in (predefined) core modules — os, path, dns, net, and fs — and by creating, exporting, and importing a user-defined module.

3. OBJECTIVE

The objectives of this experiment are:

- To understand what a module is in Node.js and why code is organised into modules.
 - To use require() to load Node's built-in core modules.
 - To write a file synchronously to disk using the fs module.
 - To read system information (platform, architecture, CPU cores, memory, current user) using the os module.
 - To build, join, and inspect file paths using the path module.
 - To resolve a domain name to an IP address using the dns module.
 - To create a TCP server and a TCP client using the net module.
 - To create a user-defined module and export multiple functions from it using module.exports.
 - To import and use a user-defined module in another file with require('./file').
-

4. THEORY

A **module** in Node.js is simply a JavaScript file whose code is isolated from other files unless it is explicitly shared. Node.js ships with a set of **built-in (core) modules** — such as `os`, `path`, `dns`, `net`, and `fs` — that are loaded with `require('moduleName')` without needing to be installed.

The `fs` (file system) module reads and writes files. `writeFileSync()` blocks execution until the file is written, which keeps the write simple for a short script.

The `os` module reports information about the machine Node.js is running on: its platform, CPU architecture, number of CPU cores, total and free memory, and the currently logged-in user.

The `path` module works with file and directory paths in a way that is safe across operating systems — joining segments together, and extracting a path's directory, file name, and extension — without the script needing to manually split strings on slashes or backslashes.

The `dns` module resolves a human-readable domain name (e.g. `google.com`) to the numeric IP address a computer actually connects to; `dns.lookup()` performs this resolution asynchronously and returns the result in a callback.

The `net` module provides raw TCP networking. `net.createServer()` creates a server that listens for incoming connections on a port, while `net.createConnection()` creates a client socket that connects to a server at a given host and port. Once connected, both sides can send data with `.write()` and receive it through the `'data'` event.

A **user-defined module** is a file created for this project rather than shipped with Node.js. Any function, object, or value assigned to `module.exports` inside that file becomes available to any other file that loads it with `require('./filename')` (the `./` tells Node to look for a local file rather than a built-in or installed package).

The key methods and concepts used in this program are:

- `require('moduleName')` loads a built-in core module by name.
- `require('./file')` loads a local, user-defined module by relative file path.
- `fs.writeFileSync(path, data)` writes data to a file, creating or overwriting it, and blocks until the write completes.
- `path.join(...segments)` combines path segments into one normalised path.
- `path.dirname()` / `path.basename()` / `path.extname()` extract the directory, file name, and extension from a path string.
- `os.platform()` / `os.arch()` / `os.cpus()` / `os.totalmem()` / `os.freemem()` / `os.userInfo()` report system information.
- `dns.lookup(hostname, callback)` resolves a domain name to an IP address asynchronously.
- `net.createServer(callback)` creates a TCP server; the callback receives a socket for each connected client.
- `net.createConnection({ port, host }, callback)` creates a TCP client socket connected to a server.
- `socket.write()` sends data over a connection; the `'data'` event receives data sent by the other side.

- `module.exports = { ... }` exposes selected functions from a file so other files can import them.

5. ALGORITHM / PROCEDURE

Part A — Predefined (Built-in) Modules

- Import the `os`, `path`, `dns`, `net`, and `fs` core modules with `require()`.
- Use `fs.writeFileSync()` to create `test.txt` containing a short message.
- Build a Windows-style file path string and a cross-platform joined path using `path.join(__dirname, ...)`.
- Log platform, architecture, CPU core count, total memory, free memory, and current user info using the `os` module.
- Log the directory name, file name, and extension of the sample file path using the `path` module.
- Call `dns.lookup()` on a domain name and log the resolved IP address and address family in its callback.
- Create a TCP server with `net.createServer()` that greets a connecting client, logs any data it receives, and logs when the client disconnects.
- Start the server listening on port 3000 and log a confirmation message.

Part B — User-Defined Module

- Create `calculator.js` containing `add()`, `subtract()`, `multiply()`, and `divide()` functions.
- Export all four functions from `calculator.js` using `module.exports`.
- Create `app.js` and import the module with `require('./calculator')`.
- Call each exported function with sample values and log the results.

Part C — TCP Client

- Create `client.js` and import the `net` module.
- Use `net.createConnection()` to connect to the server on `localhost:3000`.
- Once connected, send a message to the server with `client.write()`.
- Listen for the server's response on the `'data'` event, log it, and close the connection with `client.end()`.

6. PROGRAM

predefined_modules — server.js

```
const os = require('os');
const path = require('path');
const dns = require('dns');
const net = require('net');
const fs = require('fs');
```

```
// ===== FILE SYSTEM MODULE =====

fs.writeFileSync('test.txt', 'Hello Node.js');

// ===== PATH VARIABLES =====

const filePath =
  'C:\\CSM-B-97\\FullStack-LAB\\OBSERVATION-TASKS\\TASK-1\\Task1_Observation_Document.pdf';

const newPath = path.join(__dirname, 'data', 'config.json');

// ===== OS MODULE =====

console.log(newPath);

console.log('Operating System Info:');

console.log(`Platform: ${os.platform()}`);
console.log(`Architecture: ${os.arch()}`);
console.log(`CPU Cores: ${os.cpus().length}`);
console.log(`Total Memory: ${os.totalmem()} bytes`);
console.log(`Free Memory: ${os.freemem()} bytes`);
console.log('Current User:', os.userInfo());

// ===== PATH MODULE =====

console.log('Path Info:');

console.log(`Directory Name: ${path.dirname(filePath)}`);
console.log(`File Name: ${path.basename(filePath)}`);
console.log(`File Extension: ${path.extname(filePath)}`);
console.log(`Joined Path: ${newPath}`);

// ===== DNS MODULE =====

dns.lookup('google.com', (err, address, family) => {

  if (err) {
    console.log('DNS Lookup Error:', err.message);
    return;
  }

  console.log('DNS Info:');
  console.log(`IP Address: ${address}`);
  console.log(`IP Family: ${family}`);
});

// ===== NET MODULE =====
```

```

const server = net.createServer((socket) => {

  console.log('Client connected.');
```

```

  socket.write('Hello from the server!');
```

```

  socket.on('data', (data) => {
    console.log(`Received from client: ${data}`);
  });
```

```

  socket.on('end', () => {
    console.log('Client disconnected.');
```

```

  });
});

// Start server
server.listen(3000, () => {
  console.log('Server listening on port 3000.');
```

```

});

```

userdefined_modules — calculator.js

```

// Math Operations
function add(a, b) {
  return a + b;
}

function subtract(a, b) {
  return a - b;
}

function multiply(a, b) {
  return a * b;
}

function divide(a, b) {
  if (b === 0) throw new Error("Cannot divide by zero");
  return a / b;
}

// Export functions
module.exports = {
  add,
  subtract,
  multiply,
  divide
};

```

userdefined_modules — app.js

```
// Load user-defined module
const calc = require('./calculator');

console.log("Addition:", calc.add(15, 7));
console.log("Subtraction:", calc.subtract(50, 20));
console.log("Multiplication:", calc.multiply(6, 8));
console.log("Division:", calc.divide(100, 4));
```

client.js

```
const net = require('net');

const client = net.createConnection({
  port: 3000,
  host: 'localhost'
}, () => {

  console.log('Connected to server.');

  client.write('Hello Server');

});

client.on('data', (data) => {

  console.log(`Server says: ${data}`);

  client.end();

});
```

7. CODE EXPLANATION

Code	Explanation
<code>require('os'/'path'/'dns'/'net'/'fs')</code>	Loads a Node.js built-in core module by name.
<code>fs.writeFileSync(path, data)</code>	Writes data to a file synchronously, creating or overwriting it.
<code>path.join(__dirname, 'data', 'config.json')</code>	Builds a normalised path relative to the current file's directory.
<code>path.dirname() / basename() / extname()</code>	Extract the directory, file name, and extension from a path string.
<code>os.platform() / os.arch()</code>	Return the operating system platform and CPU architecture.
<code>os.cpus() / totalmem() / freemem()</code>	Return CPU core details and total/free system memory in bytes.
<code>os.userInfo()</code>	Returns details about the currently logged-in operating system user.

Code	Explanation
<code>dns.lookup(hostname, callback)</code>	Resolves a domain name to an IP address asynchronously and returns it in the callback.
<code>net.createServer(callback)</code>	Creates a TCP server; the callback runs once per connected client, receiving a socket.
<code>socket.write()</code> / <code>socket.on('data', ...)</code>	Sends data to the other side of a connection, or receives data sent by it.
<code>net.createConnection({ port, host }, cb)</code>	Creates a TCP client socket that connects to a server at the given host and port.
<code>module.exports = { add, subtract, ... }</code>	Exposes selected functions from calculator.js so other files can import them.
<code>require('./calculator')</code>	Loads the local calculator.js module by its relative file path.

8. TEST CASES AND EXECUTION DATA

Each program was run with Node.js and the console output was recorded below.

Test Case	Action	Expected Result	Actual Result	Status
TC-01	Run server.js	test.txt is created; OS, path, and joined-path details are logged; server starts listening on port 3000	test.txt created and all details logged correctly; "Server listening on port 3000." printed	Pass
TC-02	DNS lookup for google.com (part of server.js)	The resolved IPv4 address and address family are logged shortly after the server starts	IP address and family logged correctly a moment after startup (DNS lookup is asynchronous)	Pass
TC-03	Run app.js (userdefined_modules) independently	Addition, subtraction, multiplication, and division results are logged using the imported calculator module	All four results matched the expected arithmetic exactly	Pass
TC-04	Run client.js while server.js is already running	Client connects, sends a message, and logs the server's reply; server logs the client connecting, the message it received, and the disconnect	Client and server logs matched exactly as expected on both sides	Pass
TC-05	Run client.js while no server is running	Connection should fail since nothing is listening on port 3000	Unhandled error: connect ECONNREFUSED 127.0.0.1:3000 — the script crashes	Fail

Test Case	Action	Expected Result	Actual Result	Status
TC-06	Run server.js a second time while the first instance is still running	Second instance should not be able to bind the same port	Error: listen EADDRINUSE: address already in use 0.0.0.0:3000 — the second instance crashes	Fail (expected behaviour)

9. OUTPUT

Output 1: server.js — OS, Path, and DNS Info

```

PS D:\FullStack-LAB\02-RECORD\WEEK-6\code> node predefined_modules.js
D:\FullStack-LAB\02-RECORD\WEEK-6\code\data\config.json
Operating System Info:
Platform: win32
Architecture: x64
CPU Cores: 16
Total Memory: 16857817088 bytes
Free Memory: 2412879872 bytes
Current User: [Object: null prototype] {
  uid: -1,
  gid: -1,
  username: 'nihar',
  homedir: 'C:\\Users\\nihar',
  shell: null
}
Path Info:
Directory Name: C:\CSM-B-97\FullStack-LAB\OBSERVATION-TASKS\TASK-1
File Name: Task1_Observation_Document.pdf
File Extension: .pdf
Joined Path: D:\FullStack-LAB\02-RECORD\WEEK-6\code\data\config.json
Server listening on port 3000.
DNS Info:
IP Address: 2404:6800:4007:80e::200e
IP Family: 6
Client connected.
Received from client: Hello Server
Client disconnected.

```

Output 2: test.txt (created by fs.writeFileSync)

```
Hello Node.js
```

Output 3: app.js — User-Defined calculator Module

```

Addition: 22
Subtraction: 30
Multiplication: 48
Division: 25

```

Output 4: client.js connecting to the running server — client-side log

```

Connected to server.
Server says: Hello from the server!

```

Output 5: server.js — server-side log after the client connects

```
Client connected.  
Received from client: Hello Server  
Client disconnected.
```

10. RESULT

Thus, modules in Node.js were successfully implemented and verified. The predefined `os`, `path`, `dns`, `fs`, and `net` modules were used to write a file, read system information, resolve a domain name, and run a working TCP server and client, and a user-defined calculator module was created, exported, and imported correctly with matching results for all four operations.